neueda

The Java Collections API

Objectives

- Collections overview
- Working with Lists
- Working with Sets
- Using Iterators
- Working with Maps

Collections Overview

- The Java collections API allows you to work with groups of object references
  - **Lists** – ordered groupings of references, where duplicates are allowed
  - **Sets** – unordered groupings of references, where duplicates are not allowed
  - **Maps** – key value pairs of objects
- The collections API can be found in the **java.util** package

Primitives and Collections

- Primitives cannot be placed into collections, only object references
- **Wrapper classes** exist for each of the primitive data types
  - Byte, Short, Integer, Long
  - Float, Double
  - Char, Boolean

```
int i = 5;
Integer myInteger = new Integer(i);
int value = myInteger.intValue();
```

- Methods exist in these classes to convert from object to primitive and vice versa

Autoboxing

- Autoboxing allows you to implicitly convert between primitives and their wrapper class equivalents

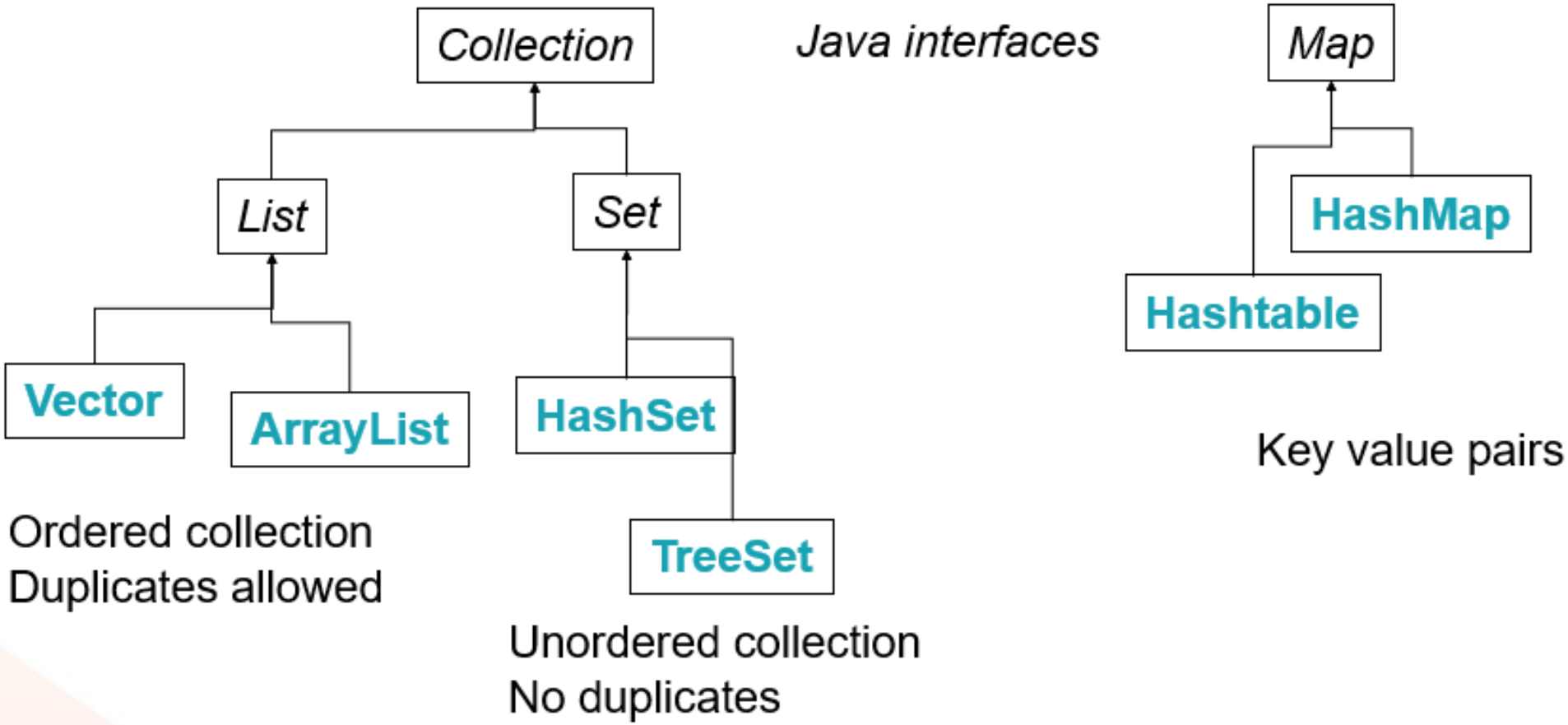
```
Integer myInteger = 3;

myInteger++;

int myInt = new Integer(2);
```

- This means that with collections you can write code to add primitives and retrieve primitives

Collections API



Creating a List

- The Collections API uses a technology called **Generics** that allows you to specify the type of object going into the collection

```
List<Person> persons = new ArrayList<Person>();
List<Person> persons = new ArrayList<>(); // Java 7 and higher
```

- The types for the reference and for the list contents are specified inside < and > signs
  - Java 7 does not require the type in the <> signs for instantiation as it is inferred from the reference
- Above, an **ArrayList of Person** objects has been created

Working with Lists

- The **List** interface provides the following operations
  - **boolean add(Object a)**
  - **<E> get(int index)**
  - **Iterator<E> iterator()**
  - **int size()**

```
List<Person> persons = new ArrayList<Person>();
for (int i=0; i<10; i++) {
  persons.add(new Person());
}
Person p = persons.get(1);
```

Types of List

- Use **ArrayList** use when data reading scenarios are more common than adding or removing data
- Use a **LinkedList** use when operations like data addition or deletion occur more frequently than reading the data
- Use a **Vector** when thread safety is required

Working with Sets

- Use a HashSet when the order does not matter
- Use a TreeSet when you want your set to be sorted into a specific order

```
Set<Person> mySet = new HashSet<>();
mySet.add(new Person("Alex", 9));
mySet.add(new Person("Abi", 7));
mySet.add(new Person("Zach", 4));
mySet.add(new Person("Emily", 2));
```

Looping using a for Loop

- No casting needs to be done when iterating over a List or Set created using generics

```
List<Person> myList = new ArrayList<Person>();
myList.add(new Person("Alex", 9));
myList.add(new Person("Abi", 7));
myList.add(new Person("Zach", 4));
myList.add(new Person("Emily", 2));

for (Person current : myList) {
    System.out.println("The name is " +  current.getName());
}
```

Looping using an Iterator

- To loop through a Collection, you can also use an **Iterator** whose methods include
  - boolean hasNext()
  - Object next() // no cast needed with generics
- The **ListIterator** also has previous() and hasPrevious()

```
Iterator<Person> iter = myList.iterator();
while (iter.hasNext()) {
   Person p = iter.next();
   System.out.println("The name is " + p.getName());
}
```

Looping with forEach() Lambda

- Since Java 8, you can iterate using a **forEach()** function

```
for (Person current : myList) {
    System.out.println("The name is " +  current.getName());
}

// becomes

myList.forEach(p -> System.out.println(“The name is ” + p.getName()));
```

Working with Predicates

- Collections can also be filtered using a Java 8 **Predicate** class
- Once a predicate has been created, it can be used to
  - Check an object to see if it matches a predicate
  - Filter a collection based on a predicate

```
Predicate<Person> menBeginningWithF = p ->
                                        p.getGender() == 'M' &&
                                        p.getName().startsWith("F");
```

Testing with Predicates

- Once I have a predicate, I can test an item against the predicate

```
if (menBeginningWithF.test(somePerson)) {
        System.out.println("This person matches!");        }
```

Filtering Collections

- Using predicates, a collection can be automatically filtered

```
people.stream().filter(menBeginningWithF).forEach(p ->
System.out.println(p.getName()));;
```

- Streams (since Java 8) allow the streaming of collections through filters and other methods
- This approach greatly simplifies how you can filter a collection and process the contents

Working with Maps

- Maps are key / value pair collections
- Use a HashMap when ordering is not important
  - To place objects into the Map, use
    - **put(<T> key, Object value)**
  - To retrieve objects from the Map, use
    - **<T> get(Object key)**

```
HashMap<String, Person> personMap = new HashMap<>();
personMap.put("Rod", new Person("Rod", 40));
personMap.put("Jane", new Person("Jane", 45));
personMap.put("Freddy", new Person("Freddy", 39));
Person p1 = personMap.get("Rod");
```

Collection Elements

- Collection elements should implement two methods
  - int hashCode()
    - Should return a unique int value for each identical object
  - boolean equals()
    - Should return true if two objects are semantically equal

```
class Person {

  public boolean equals(Object p) { … }
  public int hashCode() { … }

}
```

Sorting Collections

- Collections can be sorted, either using
  - Types that implement Comparable
  - A Collection that uses a Comparator

Using Comparable

- The items in the **TreeSet** collection must implement the **Comparable<T>** interface
  - **int compareTo (<T> obj)**

```
Set<Person> personSet = new
TreeSet<Person>();
for (int i=10; i>0; i--)  {
  Person p = new Person();
  p.setAge(i);
  personSet.add(p);
}
// sorted data  come back
for (Person p : personSet) {
   System.out.println("The age is " +  p.getAge());
}
```

```
public class Person implements
                        Comparable<Person> {
  …
  private int age;
  public int compareTo(Person otherPerson) {
       return this.age - otherPerson.age;
  }
  ..
}
```

Using a Comparator

- The **Comparator** interface is implemented by a class that is written to do the comparison
  - The Comparator is passed to the constructor of the TreeSet or TreeMap

```
Set<Person> personSet = new
TreeSet<Person>(new PersonComparator());
for (int i=10; i>0; i--)  {
  Person p = new Person();
  p.setAge(i);
  personSet.add(p);
}
// sorted data  come back
for (Person p : personSet) {
   System.out.println("The age is " +  p.getAge());
}
```

```
public class PersonComparator
        implements java.util.Comparator <Person> {
  public int compare(Person p1, Person p2) {
    return (p1.getAge() - p2.getAge());
  }
}
```

Comparator With Lambda

- A Comparator is a good candidate for using a Lambda expression

```
TreeSet<Person>( (p1,  p2) -> p1.getName().compareTo(p2.getName());
```

Summary

- Collections overview
- Working with Lists
- Working with Sets
- Using Iterators
- Working with Maps
- Sorting collections